

Evaluation of Models



1

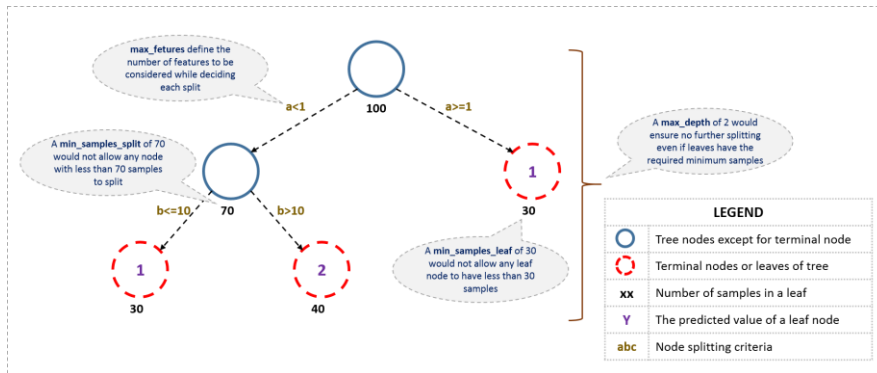
If there are more than two possible predictors, **many different decision trees** are possible for a given data set.

What makes one better than another?

- 1.Choosing the right combination of features as predictors.
- 2.Knowing when to stop splitting.



2



<https://www.analyticsvidhya.com/wp-content/uploads/2016/02/tree-infographic.png>

clf = DecisionTreeClassifier(criterion = 'gini') As seen in documentation, other parameters can be used to specify the stopping points.

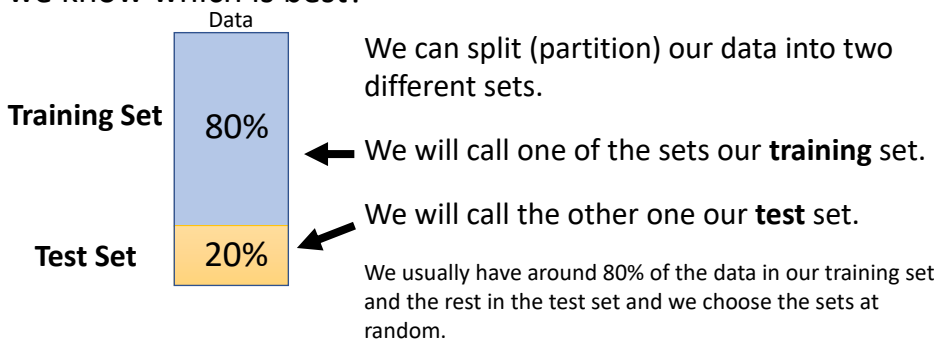
```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
max_features=None, max_leaf_nodes=None,
min_impurity_decrease=0.0, min_impurity_split=None,
min_samples_leaf=1, min_samples_split=2,
min_weight_fraction_leaf=0.0, presort=False,
random_state=None, splitter='best')
```

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

3

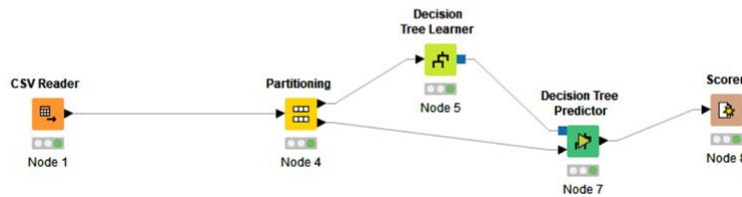
Once we have created a model, how can we evaluate it?

Given the choice between two different models, how would we know which is best?



- We **build** our model using only the **training set**.
- Then we **use** the model to make predictions for our **test data**.
- Since **we already know the value of each feature in our test data set**, we compare our **predictions** to the **actual values** and **calculate the model's accuracy**.

4



How did this model do? Let's **count how many predictions were right** and divide by the total number of data points. That will be our accuracy score.

Our Test Data

				Actual Species	Predicted Species
D sepal_length	D sepal_width	D petal_length	D petal_width	S species	S Prediction (species)
5.1	3.8	1.5	0.3	setosa	setosa
5.1	3.7	1.5	0.4	setosa	setosa
4.8	3.4	1.9	0.2	setosa	setosa
5	3	1.6	0.2	setosa	setosa
5	3.4	1.6	0.4	setosa	setosa
5.5	4.2	1.4	0.2	setosa	setosa
5.5	3.5	1.3	0.2	setosa	setosa
5.1	3.4	1.5	0.2	setosa	setosa
6.9	3.1	4.9	1.5	versicolor	virginica
5.5	2.3	4	1.3	versicolor	versicolor
5.9	3	4.2	1.5	versicolor	versicolor
6.7	3.1	4.4	1.4	versicolor	versicolor
6.1	2.8	4	1.3	versicolor	versicolor
6.8	2.8	4.8	1.4	versicolor	virginica
5.4	3	4.5	1.5	versicolor	versicolor
5.5	2.6	4.4	1.2	versicolor	versicolor

Wrong

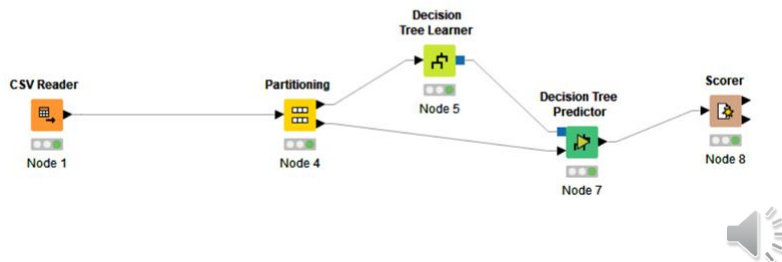
With this particular partition, our model made 28 correct predictions out of 30.

Accuracy: $28/30 = 0.933$
= 93.3%

5



We can use KNIME to create WorkFlows like this:



6

Download KNIME Analytics Platform

<https://www.knime.com/downloads>

Registration is not required. However, why not sign up for our newsletter and other updates? We only use information you provide to share more relevant content and to better understand our community. We will not share your contact information with anyone or fill your mailbox with clutter. You may unsubscribe from these emails at any time.

If you're new to the KNIME family, sign up for three introductory emails to get started quickly and access resources to help you succeed with KNIME Analytics Platform.

Email 	Company / Organization
First Name 	Last Name
Country - None -	Select your Role - None -
Select your Department - None -	Would you mind telling us how you heard about KNIME? I'd rather not tell

☐ I'd like to receive regular news updates from KNIME and accept the [privacy policy](#).

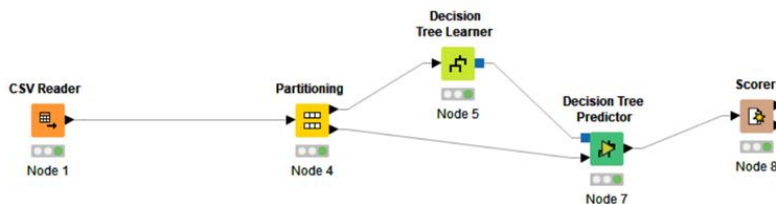
☐ In addition to the above, I would like to receive three getting started emails.

☒ I have read and accept the [terms and conditions](#) to download KNIME Analytics Platform (open source license). *

Download

7

Let's use KNIME to read in the Iris dataset, create a decision tree model, and evaluate the accuracy of the model.



Confusion Matrix

		Predicted		
Actual	Row ID	setosa	versicolor	virginica
	setosa	12	0	0
	versicolor	0	8	2
	virginica	0	0	8

With this particular partition, our model made 28 correct predictions out of 30.

Accuracy: $28/30 = 0.933$

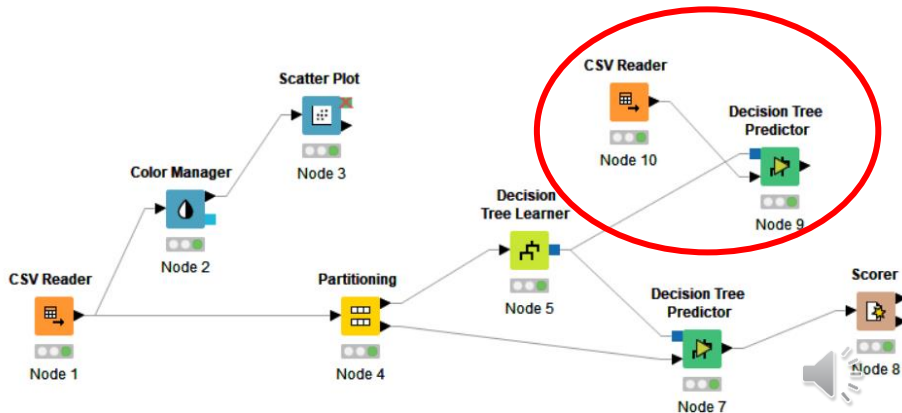
Accuracy Statistics

Row ID	TruePo...	FalsePo...	TrueNe...	FalseNe...	D Recall	D Precision	D Sensitivity	D Specificity	D F-meas...	D Accuracy	D Cohen's...
setosa	12	0	18	0	1	1	1	1	1	?	?
versicolor	8	0	20	2	0.8	1	0.8	1	0.889	?	?
virginica	8	2	20	0	1	0.8	1	0.909	0.889	?	?
Overall	?	?	?	?	?	?	?	?	?	0.933	0.899

8

If we are satisfied with our model, we can add an extra predictor node and send in some new data. Then right-click on the node and look at the Classified Data. You will find predictions there.

Predict the species of an iris that has the following measures:
 sepal_length is 0.7, sepal_width is 3.1, petal_length is 4.5, petal_width is 1.4



9

Use your KNIME Decision Tree workflow to predict the species for flowers with the following measures.

	sepal_length	sepal_width	petal_length	petal_width
Flower 1	6.7	3.1	5.9	2.3
Flower 2	5.4	2.4	4	1.3
Flower 3	4.2	3	1.1	0.1



10

Now that you see how this is done, let's do the same thing again but with Python.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

df = pd.read_csv('titanicsimplified.csv')

df['Sex'] = df['Sex'].map( {'female': 0, 'male': 1} ).astype(int)
#replace null ages with mean
df.Age = df.Age.fillna(df.Age.mean())

# Mapping Age
df.loc[ df['Age'] <= 16, 'Age'] = 0
df.loc[(df['Age'] > 16) & (df['Age'] <= 32), 'Age'] = 1
df.loc[(df['Age'] > 32) & (df['Age'] <= 48), 'Age'] = 2
df.loc[(df['Age'] > 48) & (df['Age'] <= 64), 'Age'] = 3
df.loc[ df['Age'] > 64, 'Age'] = 4

X = df.values[:, 2:4]
Y = df.values[:, 6]
Y=Y.astype('int')

clf = DecisionTreeClassifier(criterion = 'gini')

clf.fit(X,Y)
```

Earlier, we wrote some code that would create a decision tree to predict survival.

We will add some lines to split up the data into a training set and a test set and then to eventually determine the accuracy of the model.

Here is what we had before ...

```
dot_data = tree.export_graphviz(clf, out_file='TitanicDT.dot', max_depth=3, feature_names = ['Sex', 'Age'], class_names = ['Died', 'Survived'],
rounded = True, filled= True)
#use http://www.webgraphviz.com/ Open TitanicDT.dot in Notepad, copy and paste in window on website.

#Make Prediction
clf.predict([[1,4]])
```



11

Now that you see how this is done, let's do the same thing again but with Python.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
from sklearn.model_selection import train_test_split
df = pd.read_csv('titanicsimplified.csv')

df['Sex'] = df['Sex'].map( {'female': 0, 'male': 1} ).astype(int)
#replace null ages with mean
df.Age = df.Age.fillna(df.Age.mean())

# Mapping Age
df.loc[ df['Age'] <= 16, 'Age'] = 0
df.loc[(df['Age'] > 16) & (df['Age'] <= 32), 'Age'] = 1
df.loc[(df['Age'] > 32) & (df['Age'] <= 48), 'Age'] = 2
df.loc[(df['Age'] > 48) & (df['Age'] <= 64), 'Age'] = 3
df.loc[ df['Age'] > 64, 'Age'] = 4

X = df.values[:, 2:4]
Y = df.values[:, 6]
Y=Y.astype('int')

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.20, random_state=100)

clf = DecisionTreeClassifier(criterion = 'gini')

clf.fit(X_train,Y_train)
result = clf.score(X_test, Y_test)

dot_data = tree.export_graphviz(clf, out_file='TitanicDT.dot', max_depth=3, feature_names = ['Sex', 'Age'], class_names = ['Died', 'Survived'],
rounded = True, filled= True)
#use http://www.webgraphviz.com/ Open TitanicDT.dot in Notepad, copy and paste in window on website.

#Make Prediction
clf.predict([[1,4]])
```

We need another import statement for sklearn.

We need to add a line to create the train and test sets

We alter the "fit" statement to send in X_train and Y_train.

We call the "score" method and save the result.

```
In [28]: result
Out[28]: 0.7972027972027972
```



12